

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

4. Image Pre-processing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

The following preprocessing techniques were performed:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

5. Experimental Procedure

5.1 Dataset Exploration

Dataset Dimensions

Dataset	Dimensions
Training Images	(60000, 28, 28)
Training Labels	(60000,)
Testing Images	(10000, 28, 28)
Testing Labels	(10000,)

Sample Images

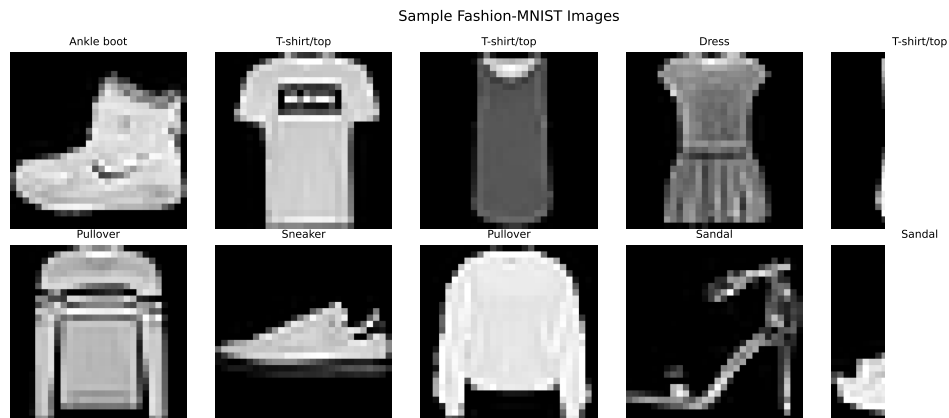


Figure 1: Sample Images from Fashion-MNIST

Observation: The sample images show 10 different black-and-white pieces of clothing in low resolution.

Class Distribution

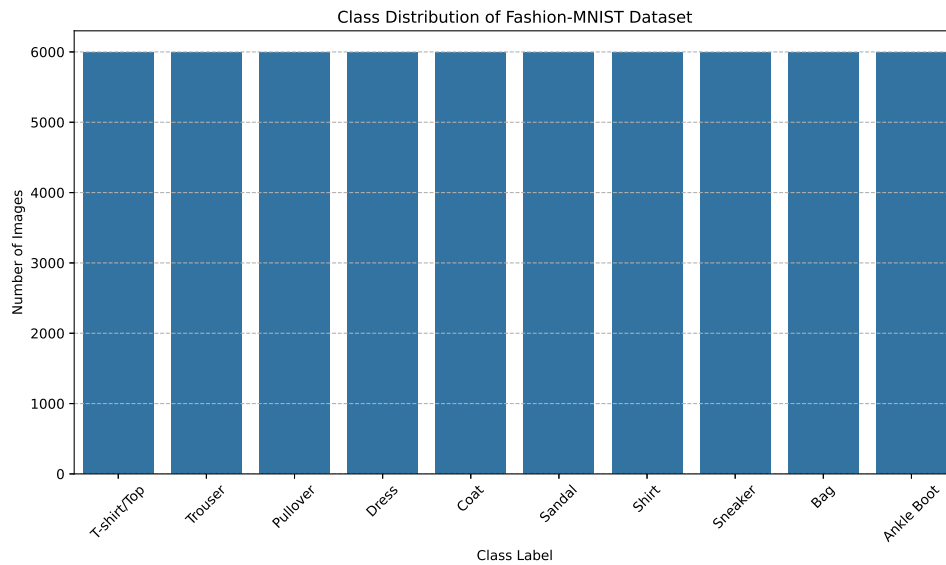


Figure 2: Class Distribution

Observation: The plot shows a uniform distribution of 6000 images of each piece of clothing, which totals to 60000 images.

5.2 Data Preprocessing

Before Preprocessing

Dataset	Dimensions
Training Images	(60000, 28, 28)
Testing Images	(10000, 28, 28)
Training Labels	(60000,)
Testing Labels	(10000,)

After Preprocessing

Dataset	Dimensions
Training Features	(60000, 784)
Testing Features	(10000, 784)
Training Labels	(60000, 10)
Testing Labels	(10000, 10)

Observation:

The images are flattened from 28×28 matrices into 784 one-dimensional feature vectors. The labels are converted into 10 one-hot encoded vectors, corresponding to the 10 classes in the Fashion-MNIST dataset.

5.3 Model Construction

Constructed an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Model Summary

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Model Parameter	Value
Total params	109,386
Trainable params	109,386
Non-trainable params	0

Observation: There are 2 hidden layers in the model, and an output layer with 10 neurons which corresponds to each of the 10 labels of the clothes. The number of trainable parameters is given by the sum of (Number of inputs x Number of neurons) + Number of biases for each neuron.

5.4 Model Training

Compiled using:

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Trained for 20 epochs with batch size 32.

Training Accuracy vs Epoch

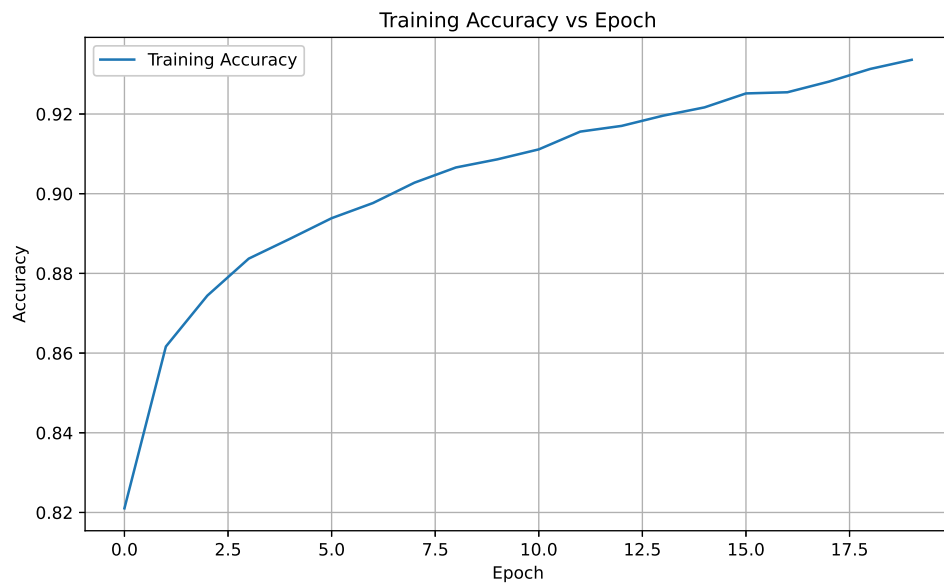


Figure 3: Training Accuracy vs Epoch

Observation: There is a sharp change in accuracy till epoch 3, then there is a gradual change in accuracy till epoch 20.

Validation Accuracy vs Epoch

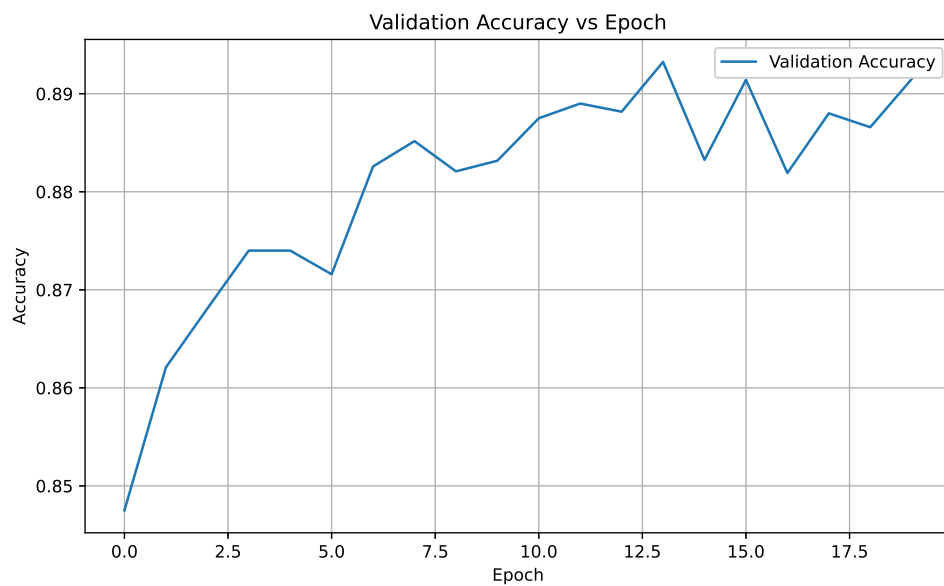


Figure 4: Validation Accuracy vs Epoch

Observation: The validation accuracy rises continually with some unstability at a few epochs.

Training Loss vs Epoch

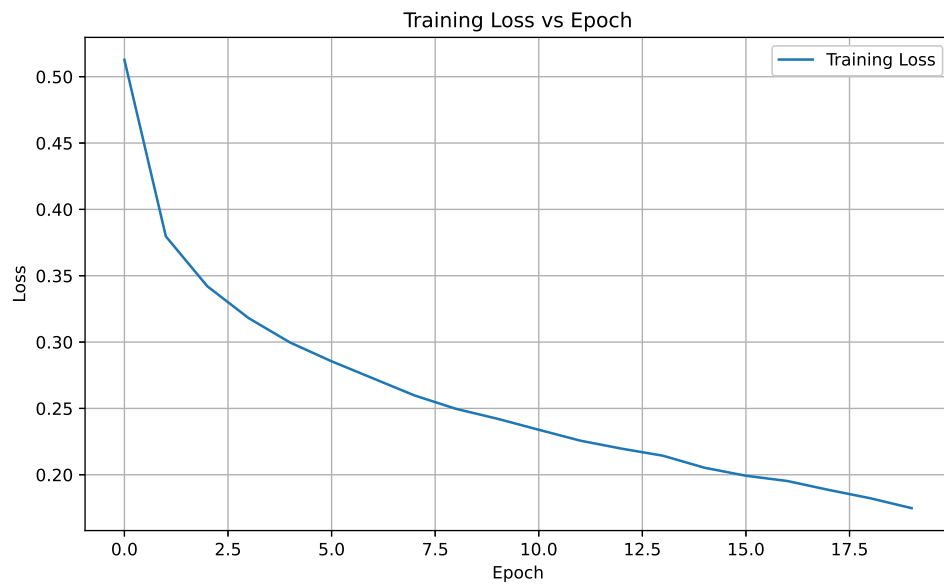


Figure 5: Training Loss vs Epoch

Observation: A sharp decline in training loss in the first few epochs, then a gradual decrease till the last epoch.

Validation Loss vs Epoch

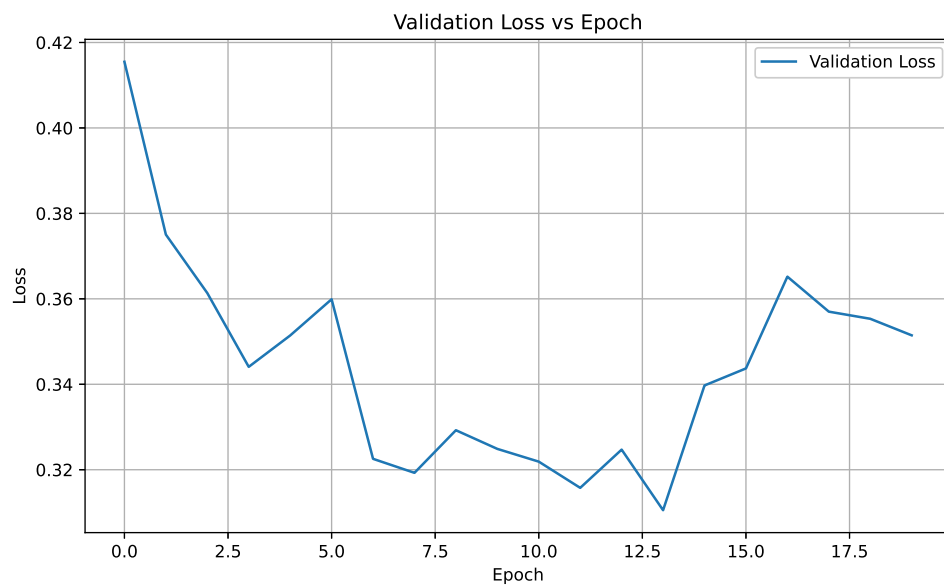


Figure 6: Validation Loss vs Epoch

Observation: The validation loss reduces till the 13th epoch, then rises till the 16th epoch, then decreases till the 20th epoch.

5.5 Model Evaluation

Performance Metrics

Metric	Value
Accuracy	0.8872
Precision	0.8880643567594608
Recall	0.8872
F1-score	0.8873841386663347

Confusion Matrix

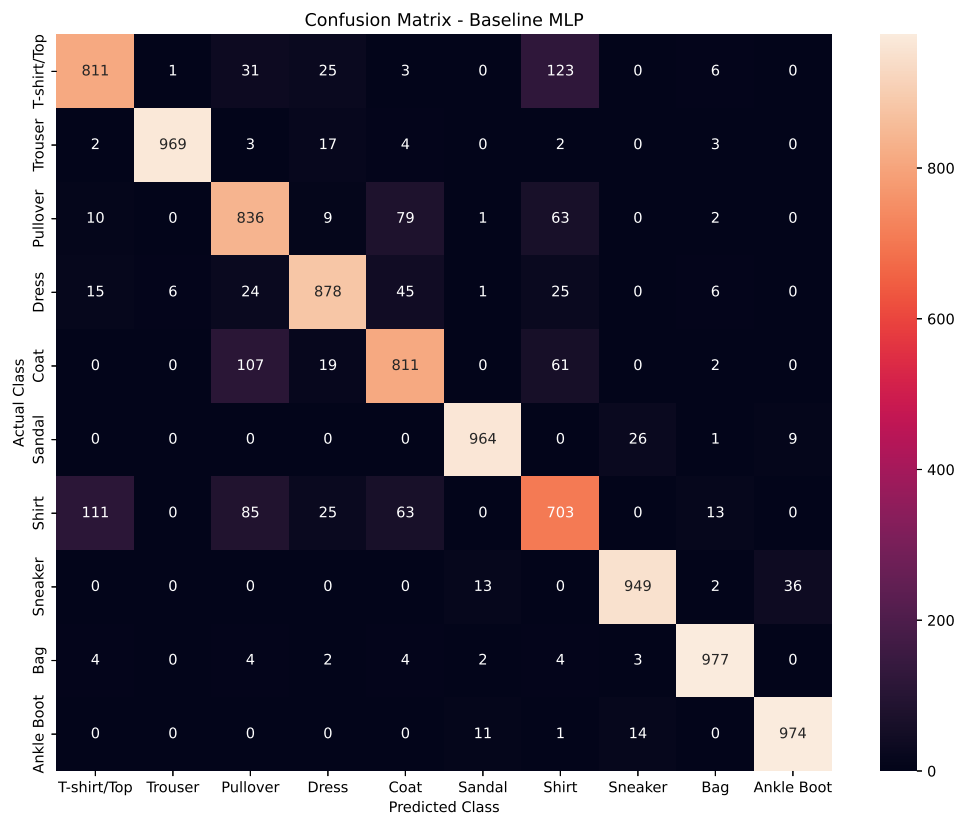


Figure 7: Confusion Matrix

Observation: Shirts were misclassified as T-shirt and Pullover too many times. T-shirts were misclassified as Shirts too many times.

Classification Report

Class	Precision	Recall	F1-score	Support
T-shirt/Top	0.85	0.81	0.83	1000
Trouser	0.99	0.97	0.98	1000
Pullover	0.77	0.84	0.80	1000
Dress	0.90	0.88	0.89	1000
Coat	0.80	0.81	0.81	1000
Sandal	0.97	0.96	0.97	1000
Shirt	0.72	0.70	0.71	1000
Sneaker	0.96	0.95	0.95	1000
Bag	0.97	0.98	0.97	1000
Ankle Boot	0.96	0.97	0.96	1000
Accuracy	0.89			10000
Macro Avg	0.89	0.89	0.89	10000
Weighted Avg	0.89	0.89	0.89	10000

Observation: Shirts and T-shirts have the least values, meaning they were misclassified more than all the other clothes.

6. Hyperparameter Optimization

Instead of manually selecting hyperparameters, automated hyperparameter optimization using **RandomizedSearchCV** together with the SciKeras wrapper was performed.

The following search space was optimized:

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

6.1 Baseline Model

Hidden Layers	2
Hidden Neurons	128,64
Learning Rate	-
Batch Size	32
Optimizer	ADAM
Activation Function	ReLU
Epochs	20
Testing Accuracy	0.8872

6.2 Hyperparameter Search

Hyperparameter Search Results

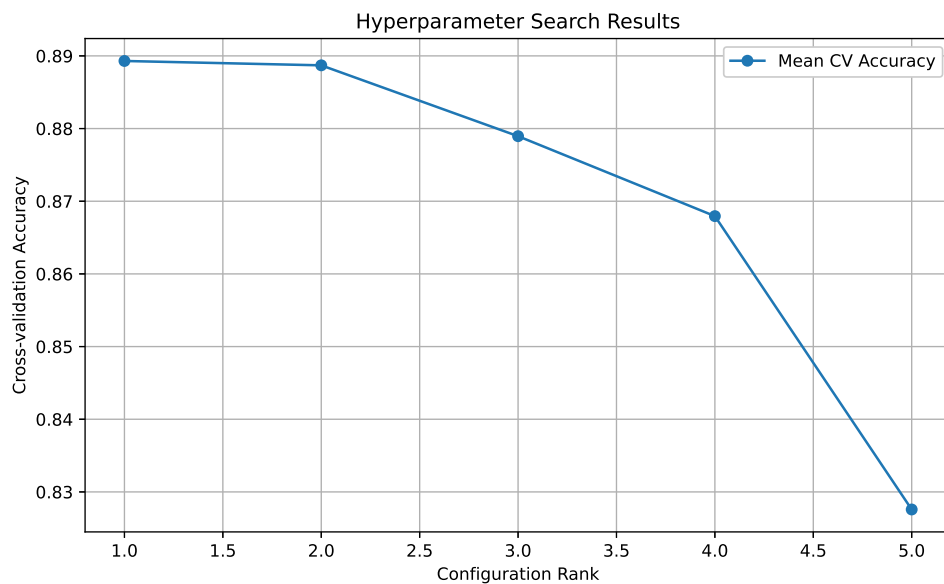


Figure 8: Hyperparameter Search Results

Observation: Cross-validation accuracy for the optimal set of hyperparameters is 0.8893. The search found that there was 1 hidden layer, 256 neurons, learning rate of 0.001, batch size of 64, ADAM optimizer, tanh activation function, 20 epochs, and dropout ratio 0.2.

6.3 Best Model

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	64
Optimizer	ADAM
Activation Function	tanh
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8893
Testing Accuracy	0.8823

6.4 Baseline vs Optimized Model

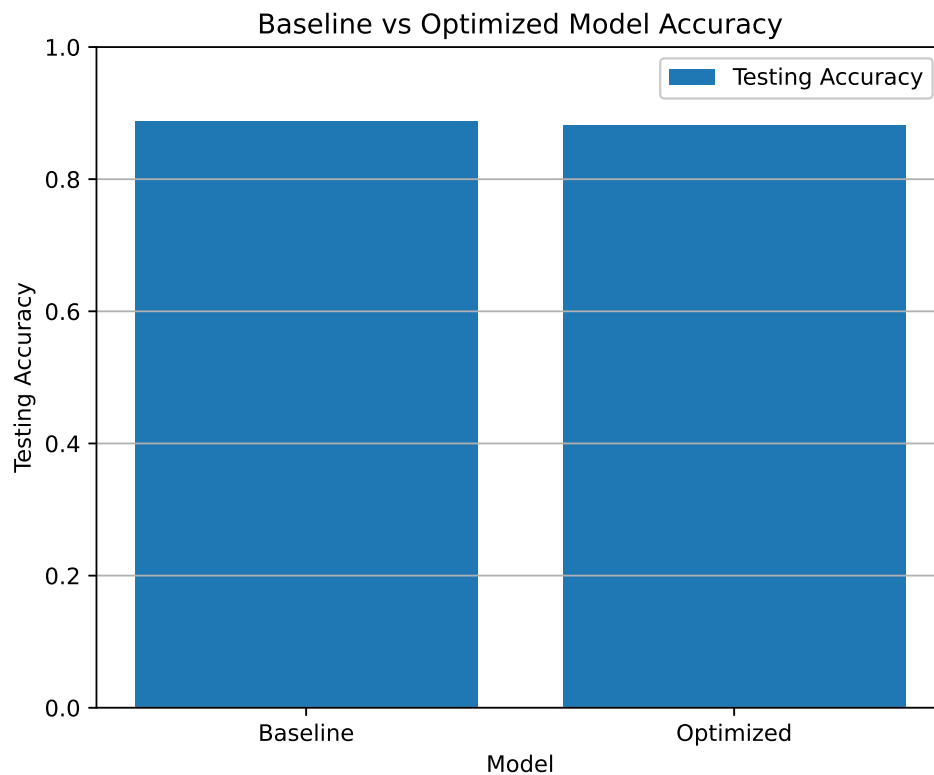


Figure 9: Best Model Accuracy Comparison

Observation:

Accuracy of the baseline model was 0.8872 and accuracy of the optimized model was 0.8823. Hence, there was no improvement in the testing accuracy of the optimized model, although training time was reduced from 111.407798 to 79.894295. There was slight drop in the precision, recall and F1-score values of the optimized model compared to the baseline model.

7. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	64
Optimizer	ADAM
Activation Function	tanh
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8893
Testing Accuracy	0.8823

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.887200	0.882300
Precision	0.888064	0.884220
Recall	0.887200	0.882300
F1-score	0.887384	0.882246
Training Time	111.407798	79.894295

8. Discussion

Answer the following:

1. Which optimization method was used?

Randomized Search using **RandomizedSearchCV** was used with the SciKeras wrapper for automated hyperparameter optimization.

2. Which hyperparameters were selected?

The hyperparameters selected were: 1 hidden layer with 256 neurons, a learning rate of 0.001, batch size of 64, Adam optimizer, tanh activation function, 20 epochs and a dropout rate of 0.2. The model achieved a cross-validation accuracy of 0.8893 and a testing accuracy of 0.8823.

3. Did optimization improve performance?

No, The optimized model achieved a testing accuracy of 0.8823 compared to 0.8872 for the baseline model. Even the precision, recall and F1-score decreased slightly. But the optimized model required less training time than the baseline model.

4. Compare Grid Search and Randomized Search.

The Grid Search method considers all combinations of the specified hyperparameters, which makes the search process exhaustive and computationally expensive. The Randomized Search method considers a specified number of random combinations of the hyperparameters, which makes it less computationally expensive. In this experiment, Randomized Search was utilized for automated hyperparameter optimization.

5. Recommend the best MLP configuration.

The optimal hyperparameters determined through the search include 1 hidden layer with 256 neurons, a learning rate of 0.001, a batch size of 64, Adam optimizer, tanh activation function, 20 iterations, and a dropout of 0.2. These parameters yielded the highest cross-validation accuracy of 0.8893.

9. Conclusion

In this experiment, a Multi-Layer Perceptron model was built for multi-class classification based on the Fashion-MNIST dataset. Images were processed through flattening from 28×28 matrices into 784-dimensional feature vectors; feature values were normalized to be within $[0,1]$, and labels were converted to one-hot encoded vectors. The baseline MLP model had two hidden layers with the ReLU activation and Adam optimizer and reached testing accuracy of 0.8872.

Then, automatic hyperparameter tuning for the model was applied with RandomizedSearchCV algorithm and the SciKeras wrapper. The best settings included 1 hidden layer with 256 neurons, learning rate of 0.001, batch size of 64, Adam optimizer, tanh activation function, 20 epochs, and 0.2 drop-out ratio. This setting provided 0.8893 cross-validation accuracy and 0.8823 testing accuracy. Although this hyperparameter tuned model did not outperform the baseline MLP in testing performance, it shortened the training time.

10. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.